

# AI: Back to Fundamentals

*(the ones you least expect)*

The screenshot shows a GitHub pull request interface. At the top, navigation tabs include Code, Issues (5), Pull requests (3), Agents, Actions, Projects, and Wiki. The pull request title is "feat: report careers changes from the opportunity sync to Slack" with a PR number "#496". A status bar indicates it is "Merged" and shows "renzit merged 4 commits into main from feat/careers-changes" with a "last week" timestamp. Below this, a summary bar shows "Conversation 3", "Commits 4", "Checks 1", and "Files changed 34", along with a diff indicator "+2100 -43".

The main content area features a comment from user "renzit" with a timestamp of "last week" and an "edited" status. The comment is structured with two sections: "What" and "How".

**What**

The opportunity sync now works out what changed on the careers site and posts it to Slack ( #alerts-mkt-tech ) — but only when something actually changed.

Marketing had no way to tell when a career appeared, disappeared, or was edited on howdylatam.com. The sync ran and left no trace of *what* changed.

**How**

**Sanity is already the previous state.** Step 2 of the sync reads every `opportunity doc before tx.commit()` , so at that moment Sanity holds the previous run's state and the ATS holds the new one. The diff is just that comparison, taken before writing. No snapshot storage, and no cold start — Sanity is already populated, so the first run produces a real diff.

The central change is widening that GROQ projection to carry every watched field, not just `_id / syncSource / status` .

On the right side, the "Reviewers" section shows that "dariomac approved these changes" with a green checkmark. Below this, sections for "Assignees", "Labels", "Projects", "Milestone", and "Development" are visible, each with a gear icon for settings. The "Development" section includes a note: "Successfully merging this pull request may close these issues."

The screenshot shows a GitHub pull request interface. At the top, navigation tabs include Code, Issues (5), Pull requests (3), Agents, Actions, Projects, and Wiki. The pull request title is "feat: report careers changes from the opportunity sync to Slack" with ID #496. A status bar indicates it is merged, with a note that 'renzit' merged 4 commits into 'main' from 'feat/careers-changes' last week. Below the title, a summary shows 3 conversations, 4 commits, 1 check, and 34 files changed, with a net change of +2100 lines and -43 deletions. A comment from 'renzit' is visible, titled "What", describing the changes to the opportunity sync and its impact on Slack notifications. The comment is followed by a "How" section detailing the implementation logic. On the right, a sidebar shows the review status: 'dariomac approved these changes' with a green checkmark. Below this, sections for Assignees, Labels, Projects, Milestone, and Development are listed, all showing 'None yet' or 'No milestone'.

<> Code Issues 5 Pull requests 3 Agents Actions Projects Wiki More ▾

Code ▾

## feat: report careers changes from the opportunity sync to Slack

#496 ✎

🔒 Merged renzit merged 4 commits into main from feat/careers-changes 📄 last week

🗨 Conversation 3 ➡ Commits 4 📄 Checks 1 📄 Files changed 34 +2100 -43 🟢🟢🟢🟢🟢

 renzit commented last week • edited ▾

### What

The opportunity sync now works out what changed on the careers site and posts it to Slack ( #alerts-mkt-tech ) — but only when something actually changed.

Marketing had no way to tell when a career appeared, disappeared, or was edited on howdylatam.com. The sync ran and left no trace of *what* changed.

### How

**Sanity is already the previous state.** Step 2 of the sync reads every `opportunity doc before tx.commit()` , so at that moment Sanity holds the previous run's state and the ATS holds the new one. The diff is just that comparison, taken before writing. No snapshot storage, and no cold start — Sanity is already populated, so the first run produces a real diff.

The central change is widening that GROQ projection to carry every watched field, not just `_id / syncSource / status` .

Reviewers

 dariomac dariomac approved these changes ✓

Assignees

No one—[assign yourself](#) ⚙️

Labels

None yet ⚙️

Projects

None yet ⚙️

Milestone

No milestone ⚙️

Development

Successfully merging this pull request may close these issues. ⚙️

```
feat/careers-changes → main
```

+2,100 -43

34 files changed

The screenshot shows a GitHub Pull Request (PR) titled "feat: report careers changes from the opportunity sync to Slack" with the number #496. The PR is merged and shows a diff of +2100 lines and -43 lines, with 34 files changed. A comment from user 'renzit' is visible, discussing the implementation of the sync. The PR is approved by 'dariomac'.

Code Issues 5 Pull requests 3 Agents Actions Projects Wiki

Code

## feat: report careers changes from the opportunity sync to Slack

#496

Merged renzit merged 4 commits into main from feat/careers-changes last week

Conversation 3 Commits 4 Checks 1 Files changed 34 +2100 -43

renzit commented last week • edited

### What

The opportunity sync now works out what changed on the careers site and posts it to Slack ( #alerts-mkt-tech ) — but only when something actually changed.

Marketing had no way to tell when a career appeared, disappeared, or was edited on howdylatam.com. The sync ran and left no trace of *what* changed.

### How

**Sanity is already the previous state.** Step 2 of the sync reads every `opportunity doc before tx.commit()`, so at that moment Sanity holds the previous run's state and the ATS holds the new one. The diff is just that comparison, taken before writing. No snapshot storage, and no cold start — Sanity is already populated, so the first run produces a real diff.

The central change is widening that GROQ projection to carry every watched field, not just `_id / syncSource / status`.

Reviewers dariomac approved these changes

Assignees No one—[assign yourself](#)

Labels None yet

Projects None yet

Milestone No milestone

Development Successfully merging this pull request may close these issues.

feat/careers-changes → main

+2,100 -43

34 files changed

✓ Approved

The screenshot shows a GitHub Pull Request (PR) interface. At the top, the navigation bar includes links for Code, Issues (5), Pull requests (3), Agents, Actions, Projects, and Wiki. The PR title is "feat: report careers changes from the opportunity sync to Slack" with the number "#496". Below the title, it says "Merged" and "renzit merged 4 commits into main from feat/careers-changes last week". The PR details show 3 conversations, 4 commits, 1 check, and 34 files changed. A comment from "renzit" is visible, discussing the changes. The right sidebar shows the "Reviewers" section with "dariomac" approved the changes. The "Assignees" section is empty, and the "Labels" section is also empty. The "Projects" section is empty, and the "Milestone" section is empty. The "Development" section shows a message: "Successfully merging this pull request may close these issues."

Code

feat: report careers changes from the opportunity sync to Slack  
#496

Merged renzit merged 4 commits into main from feat/careers-changes last week

Conversation 3 Commits 4 Checks 1 Files changed 34

renzit commented last week • edited

**What**

The opportunity sync now works out what changed on the careers site and posts it to Slack ( #alerts-mkt-tech ) — but only when something actually changed.

Marketing had no way to tell when a career appeared, disappeared, or was edited on howdylatam.com. The sync ran and left no trace of *what* changed.

**How**

**Sanity is already the previous state.** Step 2 of the sync reads every `opportunity doc before tx.commit()`, so at that moment Sanity holds the previous run's state and the ATS holds the new one. The diff is just that comparison, taken before writing. No snapshot storage, and no cold start — Sanity is already populated, so the first run produces a real diff.

The central change is widening that GROQ projection to carry every watched field, not just `_id / syncSource / status`.

Reviewers dariomac approved these changes

Assignees No one—[assign yourself](#)

Labels None yet

Projects None yet

Milestone No milestone

Development Successfully merging this pull request may close these issues.

`feat/careers-changes → main`**+2,100 -43**

34 files changed

✓ Approved



**The longer the output gets,  
the less detailed the review gets.**

**The longer the output gets,  
the less detailed the review gets.**

\* Skimmed 🤔

**The longer the output gets,  
the less detailed the review gets.**

\* Skimmed 🤔

\* Handed off to another AI (hopefully, adversarial review) 🤖

**The longer the output gets,  
the less detailed the review gets.**

\* Skimmed 🤔

\* Handed off to another AI (hopefully, adversarial review) 🤖

\* Approved almost unseen 🙈

# No answer.

**No answer.**  
**Yet.**

# Fundamentals

*the ones you least expect*

So first — the ones you do.

# Ask anyone.

## You get the same four.

University CS curricula • Cracking the Coding  
Interview • LeetCode • real job postings

Remarkably consistent. Every single source.

# Ask anyone.

## You get the same four.

University CS curricula • Cracking the Coding Interview • LeetCode • real job postings

Remarkably consistent. Every single source.



### Algorithms

Quick / Merge / *other* Sorts  
Binary / Linear / *other* Search  
Dijkstra, Dynamic  
programming, Greedy algos...

# Ask anyone.

## You get the same four.

University CS curricula • Cracking the Coding Interview • LeetCode • real job postings

Remarkably consistent. Every single source.



### Algorithms

Quick / Merge / *other* Sorts  
Binary / Linear / *other* Search  
Dijkstra, Dynamic programming, Greedy algos...



### Data structures

Arrays, Linked lists, Stacks, Queues, Trees Graphs, Hash tables, Heaps,...

# Ask anyone.

## You get the same four.

University CS curricula • Cracking the Coding Interview • LeetCode • real job postings

Remarkably consistent. Every single source.



### Algorithms

Quick / Merge / *other* Sorts  
Binary / Linear / *other* Search  
Dijkstra, Dynamic programming, Greedy algos...



### Data structures

Arrays, Linked lists, Stacks, Queues, Trees Graphs, Hash tables, Heaps,...



### Complexity / Big-O

$O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n^2)$

# Ask anyone.

## You get the same four.

University CS curricula • Cracking the Coding Interview • LeetCode • real job postings

Remarkably consistent. Every single source.



### Algorithms

Quick / Merge / *other* Sorts  
Binary / Linear / *other* Search  
Dijkstra, Dynamic programming, Greedy algos...



### Data structures

Arrays, Linked lists, Stacks, Queues, Trees Graphs, Hash tables, Heaps,...



### Complexity / Big-O

$O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n^2)$



### Boolean logic

AND, OR, NOT, XOR, De Morgan's Laws,...

**This talk *assumes* them.**

# This talk *assumes* them.

It does not replace them.

They are what makes you able to specify a problem precisely.

They are just not today's topic. Find me in the break.

Not those fundamentals, then...

# Which ones?

Not those fundamentals, then...

# Which ones?

## *Software Engineering fundamentals*

- Systematic
- Disciplined
- Quantifiable

Not those fundamentals, then...

# Which ones?

## *Software Engineering fundamentals*

- Systematic
- Disciplined
- Quantifiable





# Software Engineering

"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software."

IEEE 610.12-1990 — IEEE Standard Glossary of Software Engineering Terminology

**Hold onto those three attributes.**

**Systematic**

**Disciplined**

**Quantifiable**

## Hold onto those three attributes.

### Systematic

**A method** you can repeat,  
give to somebody else,  
and get the same result.

### Disciplined

### Quantifiable

## Hold onto those three attributes.

### Systematic

A **method** you can repeat,  
give to somebody else,  
and get the same result.

### Disciplined

You behave in a very  
controlled way, **follow**  
the defined **methods**  
and self-control to  
reach a goal.

### Quantifiable

## Hold onto those three attributes.

### Systematic

A **method** you can repeat, give to somebody else, and get the same result.

### Disciplined

You behave in a very controlled way, **follow** the defined **methods** and self-control to reach a goal.

### Quantifiable

Can be **measured**, counted, or expressed as a numerical value or amount.

Notice what the definition never mentions.

Notice what the definition never mentions.

~~Writing code.~~

Notice what the definition never mentions.

# ~~Writing code.~~

The code was never the point.

It talks about *development of software*

If there's no *writing* code...

# If there's no *writing* code...

...should you be *reading* code?

Luckly,

**Somebody** got  
here before me

**Uncle Bob Martin** ✓

@unclebobmartin



I'm significantly older than you. I started coding in the late 60s. My current strategy is to not read any of the code written by my agents. That's the only way I can take advantage of their productivity. What I do instead is to surround the agents with extreme constraints. Unit tests, gherkin tests, QA procedures, quality metrics, mutation testing, test coverage, and a plethora of others. In the end, I have very high confidence in the code they produce because they've had to run the gauntlet of all of my constraints and tests.

8:44 AM · Jul 23, 2026 · **5M** Views@unclebobmartin · [x.com/unclebobmartin/status/2080257779395154409](https://x.com/unclebobmartin/status/2080257779395154409)

**Uncle Bob Martin** ✓

@unclebobmartin



I'm significantly older than you. I started coding in the late 60s. My current strategy is to not read any of the code written by my agents. That's the only way I can take advantage of their productivity. What I do instead is to surround the agents with extreme constraints. Unit tests, gherkin tests, QA procedures, quality metrics, mutation testing, test coverage, and a plethora of others. In the end, I have very high confidence in the code they produce because they've had to run the gauntlet of all of my constraints and tests.

8:44 AM · Jul 23, 2026 · **5M** Views@unclebobmartin · [x.com/unclebobmartin/status/2080257779395154409](https://x.com/unclebobmartin/status/2080257779395154409)

# **My interpretation of Uncle Bob. Nested loops.**

**(in ninety seconds)**



# My interpretation of Uncle Bob. Nested loops. (in ninety seconds)

## OUTER LOOP — BDD

**Spec** → Gherkin · Red-Green-Refactor at the level of BEHAVIOUR

# My interpretation of Uncle Bob. Nested loops. (in ninety seconds)

## OUTER LOOP — BDD

**Spec** → Gherkin · **Red**-**Green**-Refactor at the level of BEHAVIOUR

## INNER LOOP — TDD

**Red**-**Green**-Refactor at the level of DESIGN and CODE

*the agent works in here*

# Spec-BDD-TDD loops

*The first fundamental to go back!*

Not new. Not mine. Just suddenly cheap.

# But, I won't be covering this here...

## That's covered deeply here:

[David Parry] · [ON THIS VERY STAGE]

Guardrails in Code, Not in Vibes: A Spec-First Workflow for Agents

I need the loop to make my point, so I get ninety seconds. He gets 30 minutes.

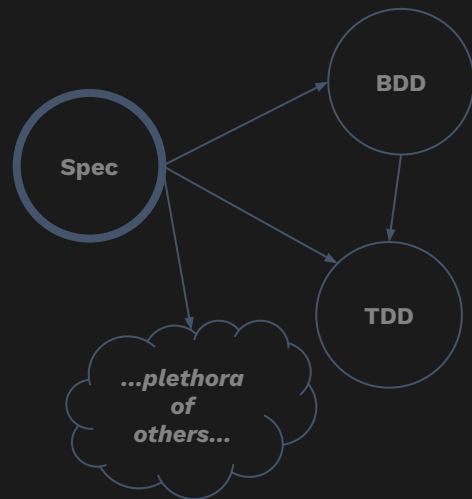
Every one of those *spec-first* tasks checks the same thing:

Every one of those *spec-first* tasks  
checks the same thing:

does the code do  
what the spec says?

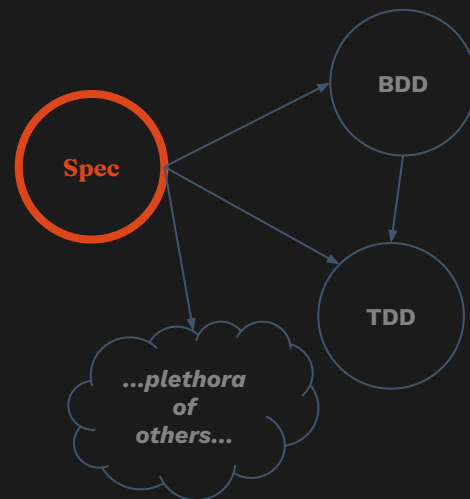
Every one of those *spec-first* tasks checks the same thing:

does the code do  
what the spec says?



Not one of them checks:

# is the spec right?



Green suite.  
Wrong product.

## And with agents it gets worse.

Agent writes the spec

Agent writes the  
Gherkin

Agent writes the code

Agent writes the tests

## And with agents it gets worse.

Agent writes the spec

Agent writes the  
Gherkin

Agent writes the code

Agent writes the tests

**The judge is grading his own work.**

## And with agents it gets worse.

Agent writes the spec

Agent writes the  
Gherkin

Agent writes the code

Agent writes the tests

**The judge is grading his own work.**

Confidence goes up. Correctness doesn't move.

# Read specs instead of code?

# Reading didn't disappear.

**Reading didn't disappear.**  
**It moved up a level of abstraction.**

code → spec

Reading didn't disappear.  
**It moved up a level of abstraction.**

code → spec



**But — do we actually know  
how to read a spec?**

# How we read specs today.

---

# How we read specs today.

---

\* Left-to-Right, Top-to-Bottom. 🙄🙄

# How we read specs today.

---

\* Left-to-Right, Top-to-Bottom. 🙄

\* "Looks fine to me." 👍

# How we read specs today.

---

\* Left-to-Right, Top-to-Bottom. 🙄

\* "Looks fine to me." 👍

\* Approve. ✅

# How we read specs today.

---

\* Left-to-Right, Top-to-Bottom. 🙄

\* "Looks fine to me." 👍

\* Approve. ✅

Back to slide 2 

**We need a “better way of reading”**

**We need a “better way of reading”**

**We need a more “Engineering” way of reading.**

**We need a “better way of reading”**

**We need a more “Engineering” way of reading.**



# **We solved this before most of us were writing code.**

---

# We solved this before most of us were writing code.



## Fagan

Fagan inspections —  
IBM. Formal,  
role-based, measured.

1976

# We solved this before most of us were writing code.



## Fagan

Fagan inspections —  
IBM. Formal,  
role-based, measured.

1976



## PBR

Perspective-Based  
Reading — Basili et  
al., with NASA/GSFC.

1994–95

# We solved this before most of us were writing code.



## Fagan

Fagan inspections —  
IBM. Formal,  
role-based, measured.

1976



## PBR

Perspective-Based  
Reading — Basili et  
al., with NASA/GSFC.

1994–95



## AGENTIC ERA

We are again trusting  
heavily in our Specs  
and documentation...  
we need those old  
Fundamentals again!

2026

# Perspective-Based Reading (PBR)

**The one you least expect.**

**You don't read it. You produce something from it.**

**You don't read it. You produce something from it.**

**TESTER**

*produces*

**the test cases**

**You don't read it. You produce something from it.**

**TESTER**

*produces*

**the test cases**

**DESIGNER (DEVELOPER)**

*produces*

**the high-level design**

# You don't read it. You produce something from it.

**TESTER**

*produces*

**the test cases**

**DESIGNER (DEVELOPER)**

*produces*

**the high-level design**

**USER**

*produces*

**the documentation**

Then, answer the guiding questions: completeness, contradictions, correctness, ambiguity.

Reading lets you ignore a gap.  
**Producing makes the gap visible.**

Reading lets you ignore a gap.  
**Producing makes the gap visible.**

You can read "Show the user's name" and nod.

# Reading lets you ignore a gap. Producing makes the gap visible.

You can read "Show the user's name" and nod.

You cannot write `expected_text = ?` and nod.

# Before someone does this to me later:

## Validity threats

- | The original experiments had small samples.
- | They ran inside the NASA SEL environment — generalisation is an open question.
- | Gains were clearer at team level than individual level.
- | A later paper is titled "Are the Perspectives Really Different?" — the core assumption has been challenged.

# Perspective-Based Reading (PBR)

An example of it

#### REQ-14 — Referral bonus

When a user refers a friend and the friend completes their first purchase, the referring user receives a 10% discount on their next purchase. The discount is applied automatically at checkout. Referral bonuses expire after 30 days.

#### REQ-14 — Referral bonus

When a user refers a friend and the friend completes their first purchase, the referring user receives a 10% discount on their next purchase. The discount is applied automatically at checkout. Referral bonuses expire after 30 days.

**Hands up — is this a good requirement?**

# The tester's artifact, as far as it got.

| # | GIVEN | WHEN | THEN — EXPECTED RESULT |
|---|-------|------|------------------------|
|---|-------|------|------------------------|

---

## The tester's artifact, as far as it got.

| # | GIVEN                                      | WHEN                    | THEN — EXPECTED RESULT                  |
|---|--------------------------------------------|-------------------------|-----------------------------------------|
| 1 | Ana referred Bob; Bob has no prior orders. | Bob completes checkout. | Ana's referral bonus becomes available. |

## The tester's artifact, as far as it got.

| # | GIVEN                                            | WHEN                    | THEN — EXPECTED RESULT                  |
|---|--------------------------------------------------|-------------------------|-----------------------------------------|
| 1 | Ana referred Bob; Bob has no prior orders.       | Bob completes checkout. | Ana's referral bonus becomes available. |
| 2 | Ana referred Bob; Bob already has 1 prior order. | Bob completes checkout. | No bonus is granted.                    |

## The tester's artifact, as far as it got.

| # | GIVEN                                            | WHEN                                                           | THEN — EXPECTED RESULT                  |
|---|--------------------------------------------------|----------------------------------------------------------------|-----------------------------------------|
| 1 | Ana referred Bob; Bob has no prior orders.       | Bob completes checkout.                                        | Ana's referral bonus becomes available. |
| 2 | Ana referred Bob; Bob already has 1 prior order. | Bob completes checkout.                                        | No bonus is granted.                    |
| 3 | Ana has an available bonus.                      | Ana do the checkout: \$100.00 + \$21.00 tax + \$10.00 shipping |                                         |

## The tester's artifact, as far as it got.

| # | GIVEN                                            | WHEN                                                           | THEN — EXPECTED RESULT                                      |
|---|--------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------|
| 1 | Ana referred Bob; Bob has no prior orders.       | Bob completes checkout.                                        | Ana's referral bonus becomes available.                     |
| 2 | Ana referred Bob; Bob already has 1 prior order. | Bob completes checkout.                                        | No bonus is granted.                                        |
| 3 | Ana has an available bonus.                      | Ana do the checkout: \$100.00 + \$21.00 tax + \$10.00 shipping | <b>STOPPED — 10% of what? \$10.00, \$12.10, or \$13.10?</b> |

## The tester's artifact, as far as it got.

| # | GIVEN                                            | WHEN                                                           | THEN — EXPECTED RESULT                                                                  |
|---|--------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 1 | Ana referred Bob; Bob has no prior orders.       | Bob completes checkout.                                        | Ana's referral bonus becomes available.                                                 |
| 2 | Ana referred Bob; Bob already has 1 prior order. | Bob completes checkout.                                        | No bonus is granted.                                                                    |
| 3 | Ana has an available bonus                       | Ana do the checkout: \$100.00 + \$21.00 tax + \$10.00 shipping | <b>STOPPED — 10% of what? \$10.00, \$12.10, or \$13.10?</b>                             |
| 4 | Ana has an available bonus.                      | 30 days pass                                                   | <b>STOPPED — 30 days from the referral, B's purchase, or the grant? Whose timezone?</b> |
| 5 | Ana already redeemed the bonus                   | Bob refunds their first purchase                               | <b>STOPPED — the spec is silent</b>                                                     |
| 6 | Ana has a bonus and a 20% seasonal coupon        | Ana checks out                                                 | <b>STOPPED — do they stack? Which applies first?</b>                                    |

# Two rows.

# Two rows.

I got two rows in before I had to go and ask somebody.

# Two rows.

I got two rows in before I had to go and ask somebody.

And nobody put their hand down thirty seconds ago — including me, and I wrote this requirement.

# Designer perspective

**Same spec.**

# Designer perspective

**Same spec.**



**Save the credit somewhere, or evaluate the rule at checkout?** The spec does not decide, and it changes the schema.

# Designer perspective

Same spec.



Save the credit somewhere, or evaluate the rule at **checkout**? The spec does not decide, and it changes the schema.



There is no **referral entity** — should I save the discount to the order, to the buyer... (there is no state to design against).

# Designer perspective

Same spec.



**Save the credit somewhere, or evaluate the rule at checkout?** The spec does not decide, and it changes the schema.



There is no **referral entity** — should I save the discount to the order, to the buyer... (there is no state to design against).



**What if two referrals are completed in the same instant?** Do they stack, compound, overwrite, or queue?

# Designer perspective

Same spec.



**Save the credit somewhere, or evaluate the rule at checkout?** The spec does not decide, and it changes the schema.



There is no **referral entity** — should I save the discount to the order, to the buyer... (there is no state to design against).



**What if two referrals are completed in the same instant?** Do they stack, compound, overwrite, or queue?



The **purchase-completed event arrives twice**. What makes granting idempotent?

# User perspective

**Same spec.**

# User perspective

**Same spec.**



How does the user even find out they have a **bonus**? Nothing requires a **notification**.

# User perspective

Same spec.



How does the user even find out they have a **bonus**? Nothing requires a **notification**.



Can they **see** the bonus **before checkout**, or does it just appear?

# User perspective

Same spec.



How does the user even find out they have a **bonus**? Nothing requires a **notification**.



Can they **see** the bonus **before checkout**, or does it just appear?



**What does Bob get?**  
Nothing? Deliberate, or an omission?

# User perspective

Same spec.



How does the user even find out they have a **bonus**? Nothing requires a **notification**.



Can they **see** the bonus **before checkout**, or does it just appear?



**What does Bob get?**  
Nothing? Deliberate, or an omission?



**Can I refer myself** from a second account? The spec is silent on abuse.

---

3

Sentences

---

3

Sentences

14

Questions

---

3

Sentences

14

Questions

0

Coding defects

No unit test, no mutation score, no coverage number finds any of them.

## The Gherkin my agent actually produced.

**Feature:** Referral bonus

**Scenario:** Referrer receives discount after friend's first purchase

**Given** user **Ana** has referred user **Bob**

And user **Bob** has no previous orders

**When** user **Bob** completes a purchase of \$100.00

**Then** user **Ana** receives a discount of \$10.00 on their next order

And the discount expires 30 days later

**AI answered all fourteen questions.**  
**It never asked you one.**

**Green suite. Wrong product.**

# So why did we stop doing this?

## So why did we stop doing this?

# Cost.

Three trained readers × hours each × a reconciliation meeting  
— **per requirement.**

Not because it didn't work. Because nobody could afford it.

## And now?

**Seconds**

not hours

**Parallel**

not sequential

**No calendar**

no meeting

The practice didn't improve.

**It got affordable.**

# Perspectives nobody could staff in 1995.

---

**Security**

---

the threat model

---

**Software Reliability Engineer**

---

the runbook

---

**Fraud**

---

the attack plan

---

**Privacy**

---

the data inventory

---

**Accessibility**

---

the keyboard walkthrough

---

**i18n**

---

the same flow, another locale

---

**Cost to serve**

---

the unit economics

---

**Legal**

---

the disclosure text

Every one of these used to be a specialist and a meeting. Now it's a file.

# All of it is in a repo.

Both of them pinned at <https://github.com/dariomac>

pbr-for-specs



agentic-pbr



Run it on your own spec tonight. It takes about a minute.

**Agents find ambiguity.**  
**Humans resolve it.**

## Where it plugs in.



Red-Green-Refactor, applied to the spec itself — before a single line of Gherkin exists.

feat/careers-changes → main

+2,100 -43

✓ Approved

feat/careers-changes → main

+2,100 -43

✓ Approved

# I still don't read it.

I'm not reading less than I used to.  
**I'm reading *different*.**

A smaller artifact. Far more carefully. With a technique.





**AI didn't make the  
fundamentals obsolete.**

**AI didn't make the  
fundamentals obsolete.  
It made them affordable.**

# AI: Back to Fundamentals

*(the ones you least expect)*



# AI: Back to Fundamentals

*(the ones you least expect)*



Thank  
you!